

from overfitting: it's good at predicting the past and almost exactly as good at predicting the future. Quite often, instead of getting this good news, we get bad news—that our model is worse than we thought it was—but it's good to get this news because we can still make improvements before we start using our model in a real scenario. In such cases, before our model is ready to be deployed in real life, we'll need to make improvements to it so that its error rate *on the test set* is minimized.

Improvements and Refinements

You may find that you've created a decision tree that has lower accuracy than you would like. For example, you might have worse accuracy than you should because you're guilty of overfitting. Many of the strategies for dealing with overfitting issues boil down to some kind of simplification, since simple machine learning models are less likely to suffer from overfitting than are complex models.

The first and easiest way to simplify our decision tree models is to limit their maximum depth; since depth is a variable that we can redefine in one short line, this is easy to do. To determine the right depth, we have to check the error rates on out-of-sample data for different depths. If the depth is too high, it's likely to cause high error because of overfitting. If the depth is too low, it is likely to cause high error because of *underfitting*. You can think of underfitting as something like the mirror image of overfitting. Overfitting consists of attempting to learn from patterns that are arbitrary or irrelevant—in other words, learning “too much” from noise in our training data, like whether someone is wearing a green shirt. Underfitting consists of failing to learn enough—creating models that miss crucial patterns in the data, like whether someone is obese or uses tobacco.

Overfitting tends to result from models that have too many variables or too much depth, whereas underfitting tends to result from models that have too few variables or too little depth. Just as with many situations in algorithm design, the right place to be is a happy medium between too high and too low. Choosing the right parameters for a machine learning model, including the depth of a decision tree, is often referred to as *tuning*, because fixing the tightness of a string on a guitar or violin also relies on finding a happy medium between a pitch that's too high and one that's too low.